



Fine Tuning and RAG



Foundation Models and Fine-Tuning

What are Foundation Models?

Foundation models are large-scale AI models pre-trained on vast, diverse datasets. These models are designed to be highly generalizable, providing the base for a wide range of downstream tasks. Examples of foundation models include GPT-4, BERT, and CLIP. These models are versatile, capable of handling various data types such as text, images, and audio.

Fine-Tuning

Fine-tuning refers to the process of customizing a pre-trained foundation model for a specific task by training it on additional, domain-specific data. This approach allows organizations to adapt foundation models to their needs without the need for extensive retraining from scratch. For example, a general language model like GPT can be fine-tuned to better handle finance-related text, resulting in models like FinBERT.

Benefits of Fine-Tuning

- **Customization:** Fine-tuning enhances the model's performance for specific applications, making it more relevant for niche use cases.
- **Efficiency:** It requires less computational power and data compared to training a model from scratch.
- **Scalability:** Fine-tuned models can be deployed across various domains, from healthcare to finance and beyond.

Fine-Tuning – Getting started

Create fine-tuning job

POST `https://api.openai.com/v1/fine_tuning/jobs`

Creates a fine-tuning job which begins the process of creating a new model from a given dataset.

Response includes details of the enqueued job including job status and the name of the fine-tuned models once complete.

[Learn more about fine-tuning](#)

Request body

model string **Required**

The name of the model to fine-tune. You can select one of the [supported models](#).

training_file string **Required**

The ID of an uploaded file that contains training data.

See [upload file](#) for how to upload a file.

Your dataset must be formatted as a JSONL file. Additionally, you must upload your file with the purpose `fine-tune`.

The contents of the file should differ depending on if the model uses the [chat](#) or [completions](#) format.

See the [fine-tuning guide](#) for more details.

Default

Epochs

Validation file

W&B Integration

Example request

curl 

```
1 curl https://api.openai.com/v1/fine_tuning/jobs \
2   -H "Content-Type: application/json" \
3   -H "Authorization: Bearer $OPENAI_API_KEY" \
4   -d '{
5     "training_file": "file-BK7bzQj3FfZFXr7DbL6xJwfo",
6     "model": "gpt-4o-mini"
7   }'
```

Response



```
1 {
2   "object": "fine_tuning.job",
3   "id": "ftjob-abc123",
4   "model": "gpt-4o-mini-2024-07-18",
5   "created_at": 1721764800,
6   "fine_tuned_model": null,
7   "organization_id": "org-123",
8   "result_files": [],
9   "status": "queued",
10  "validation_file": null,
```

<https://platform.openai.com/docs/api-reference/fine-tuning/create>

Retrieval-Augmented Generation (RAG)

What is RAG?

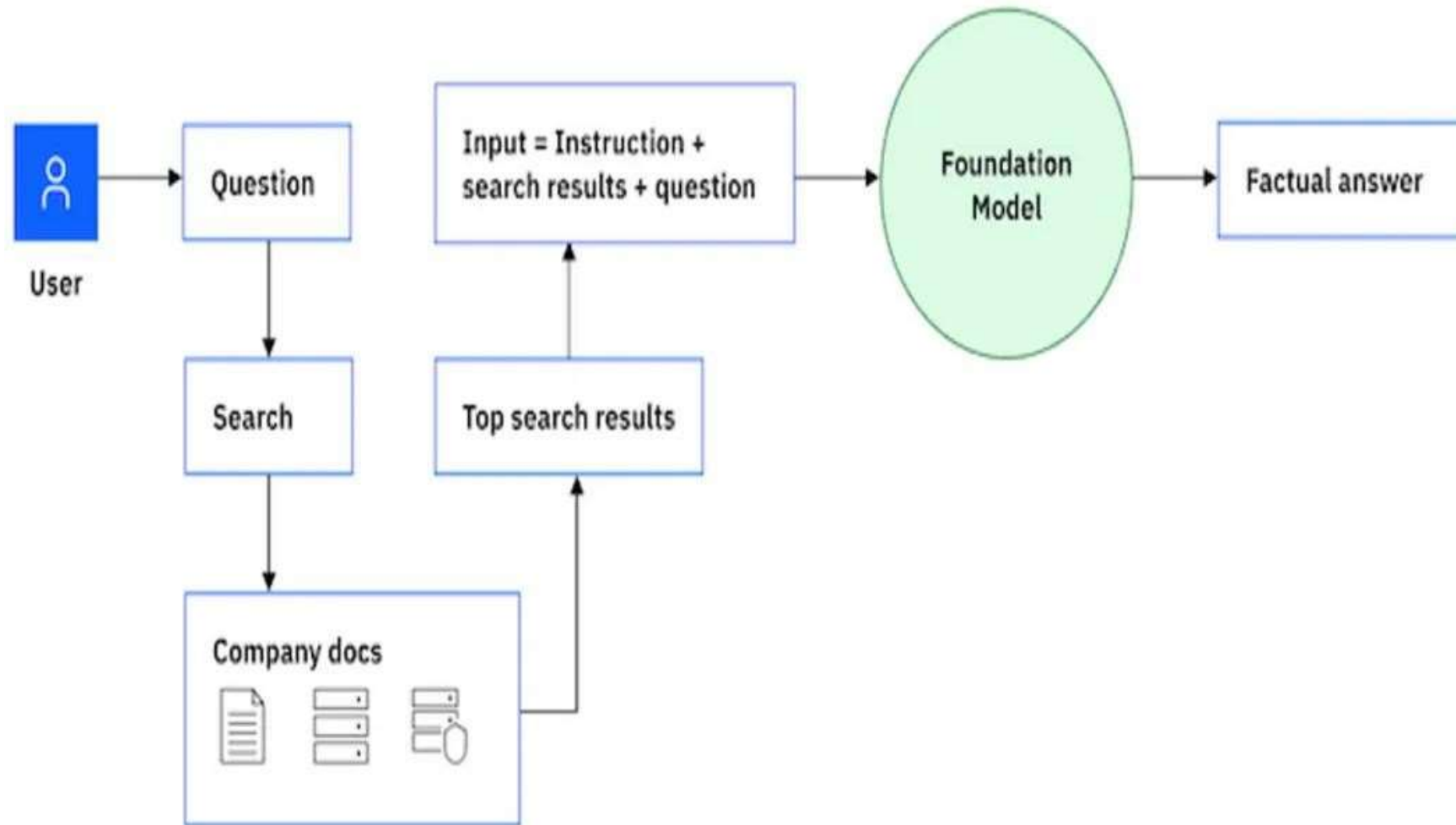
Retrieval-Augmented Generation (RAG) enhances a language model's capabilities by allowing it to access external data sources in real-time. It works by retrieving relevant information from databases or other sources and combining it with the user's query to generate a more contextually accurate and up-to-date response. This is particularly useful when dealing with constantly evolving fields like healthcare or finance, where accessing the latest information is crucial.

RAG VS. Fine tuning

RAG differs from fine-tuning primarily in how each method handles information:

- **RAG** dynamically retrieves external data from databases or APIs during inference, enabling the model to access and incorporate real-time, ensuring the generated responses are up-to-date. It doesn't alter the model's internal knowledge but enhances its answers with current, relevant data.
- In contrast, **fine-tuning** involves retraining the model with a domain-specific dataset to embed specialized knowledge directly into the model itself. This means fine-tuned models provide consistent, specialized outputs but lack the ability to adapt to new or updated information without retraining. This permanently embeds domain-specific knowledge within the model, making it more adept at particular tasks but unable to incorporate new information unless retrained

Retrieval-Augmented Generation (RAG) – Cont.

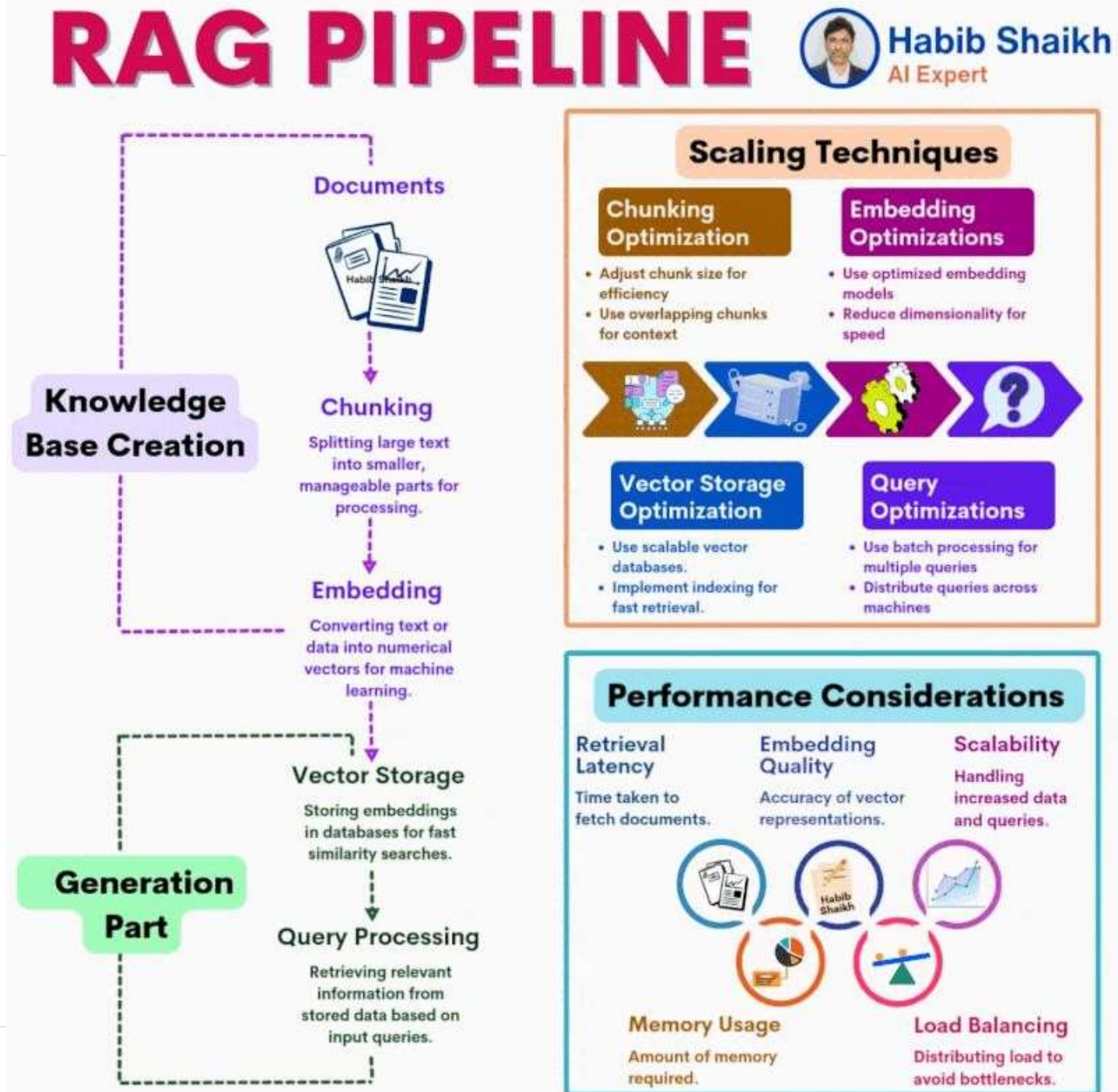


Source: IBM Developer. (n.d.). <https://developer.ibm.com/articles/awb-retrieval-augmented-generation-with-langchain-and-elastic-db/>

RAG data pipeline flow

High-level flow for a data pipeline that supplies grounding data for a RAG application:

1. Documents are either pushed or pulled into a data pipeline.
2. The data pipeline processes each document individually by completing the following steps:
 - a. **Chunk document:** Breaks down the document into semantically relevant parts that ideally have a single idea or concept.
 - b. **Enrich chunks:** Adds metadata fields that the pipeline creates based on the content in the chunks. The data pipeline categorizes the metadata into discrete fields, such as title, summary, and keywords.
 - c. **Embed chunks:** Uses an embedding model to vectorize the chunk and any other metadata fields that are used for vector searches.
 - d. **Persist chunks:** Stores the chunks in the search index.



RAG – Embeddings

Embeddings are a way to represent words, sentences, or even entire documents as dense vectors in a high-dimensional space.

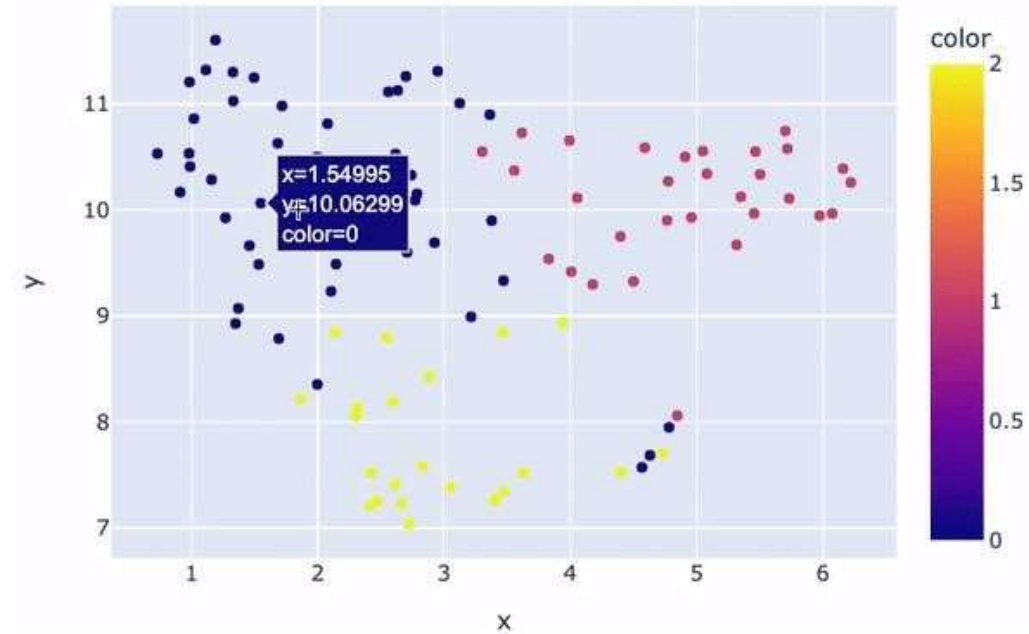
An embedding is a mathematical representation of an object, such as text. When a neural network is being trained, many representations of an object are created. Each representation has connections to other objects in the network. *The purpose of embeddings is to capture the semantic meaning of text, such that words or phrases with similar meanings are located closer to each other in this vector space.*

For example:

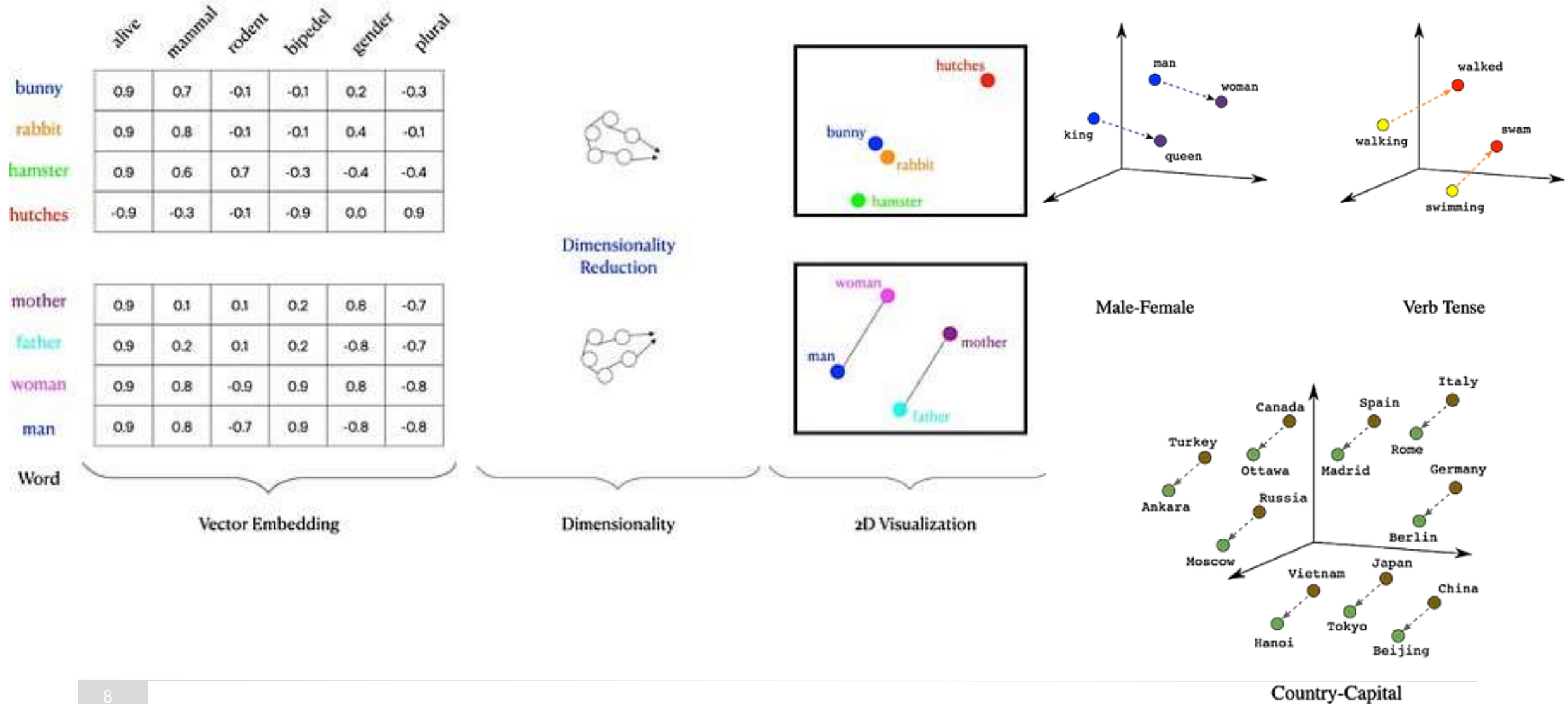
- The words “king” and “queen” might be close to each other in an embedding space because they share similar semantics (both are royalty).

Embedding similarity: the distance between any two items can be calculated mathematically and can be interpreted as a measure of relative similarity between those two items.

The AI model is trained in such a way that these vectors capture the essential features and characteristics of the underlying data.



RAG – Embeddings (Cont.)



RAG – Creating Embeddings

Create embeddings

POST `https://api.openai.com/v1/embeddings`

Creates an embedding vector representing the input text.

Request body

input string or array **Required**

Input text to embed, encoded as a string or array of tokens. To embed multiple inputs in a single request, pass an array of strings or array of token arrays. The input must not exceed the max input tokens for the model (8192 tokens for `text-embedding-ada-002`), cannot be an empty string, and any array must be 2048 dimensions or less. [Example Python code](#) for counting tokens. Some models may also impose a limit on total number of tokens summed across inputs.

✓ Show possible types

model string **Required**

ID of the model to use. You can use the [List models](#) API to see all of your available models, or see our [Model overview](#) for descriptions of them.

dimensions integer **Optional**

The number of dimensions the resulting output embeddings should have. Only supported in `text-embedding-3` and later models.

encoding_format string **Optional** Defaults to float

The format to return the embeddings in. Can be either `float` or `base64`.

Example request

curl ↕

```
1 curl https://api.openai.com/v1/embeddings \
2   -H "Authorization: Bearer $OPENAI_API_KEY" \
3   -H "Content-Type: application/json" \
4   -d '{
5     "input": "The food was delicious and the waiter...",
6     "model": "text-embedding-ada-002",
7     "encoding_format": "float"
8   }'
```

Response

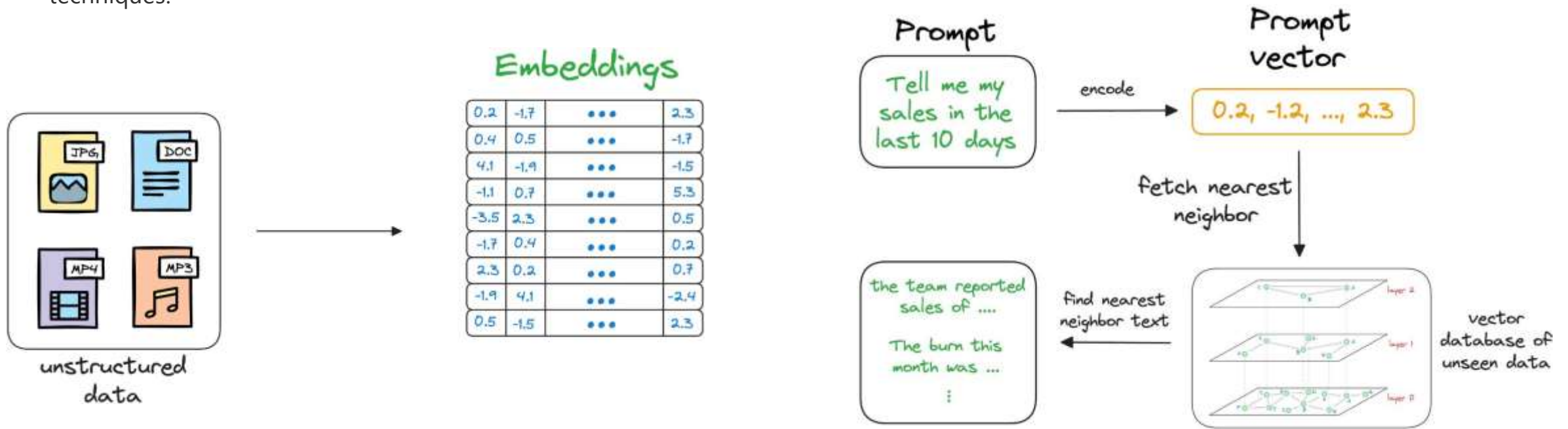
↗

```
1 {
2   "object": "list",
3   "data": [
4     {
5       "object": "embedding",
6       "embedding": [
7         0.0023064255,
8         -0.009327292,
9         .... (1536 floats total for ada-002)
10        -0.0028842222,
11      ],
12       "index": 0
13     }
14   ],
```

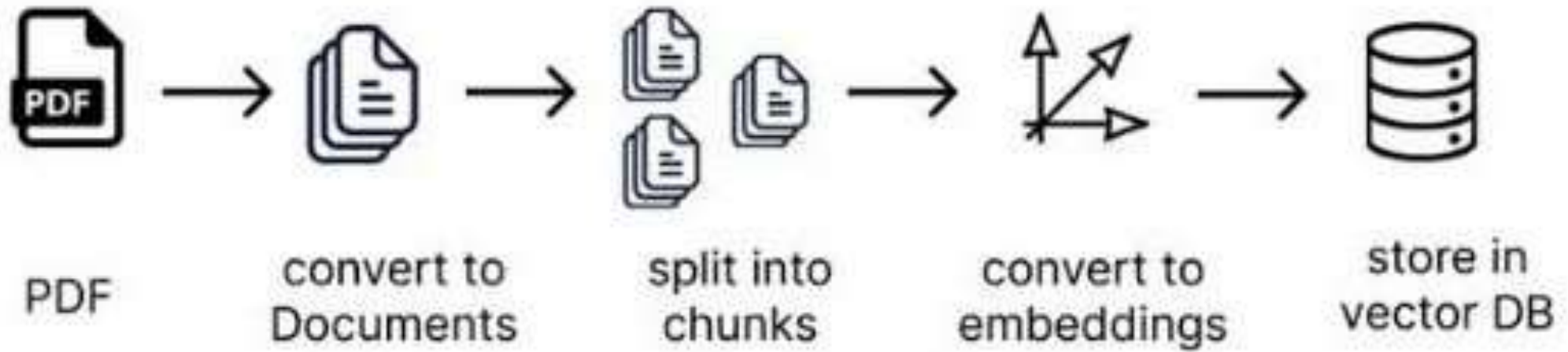
RAG – Vector databases

Vector database stores unstructured data (text, images, audio, video, etc.) in the form of vector embeddings.

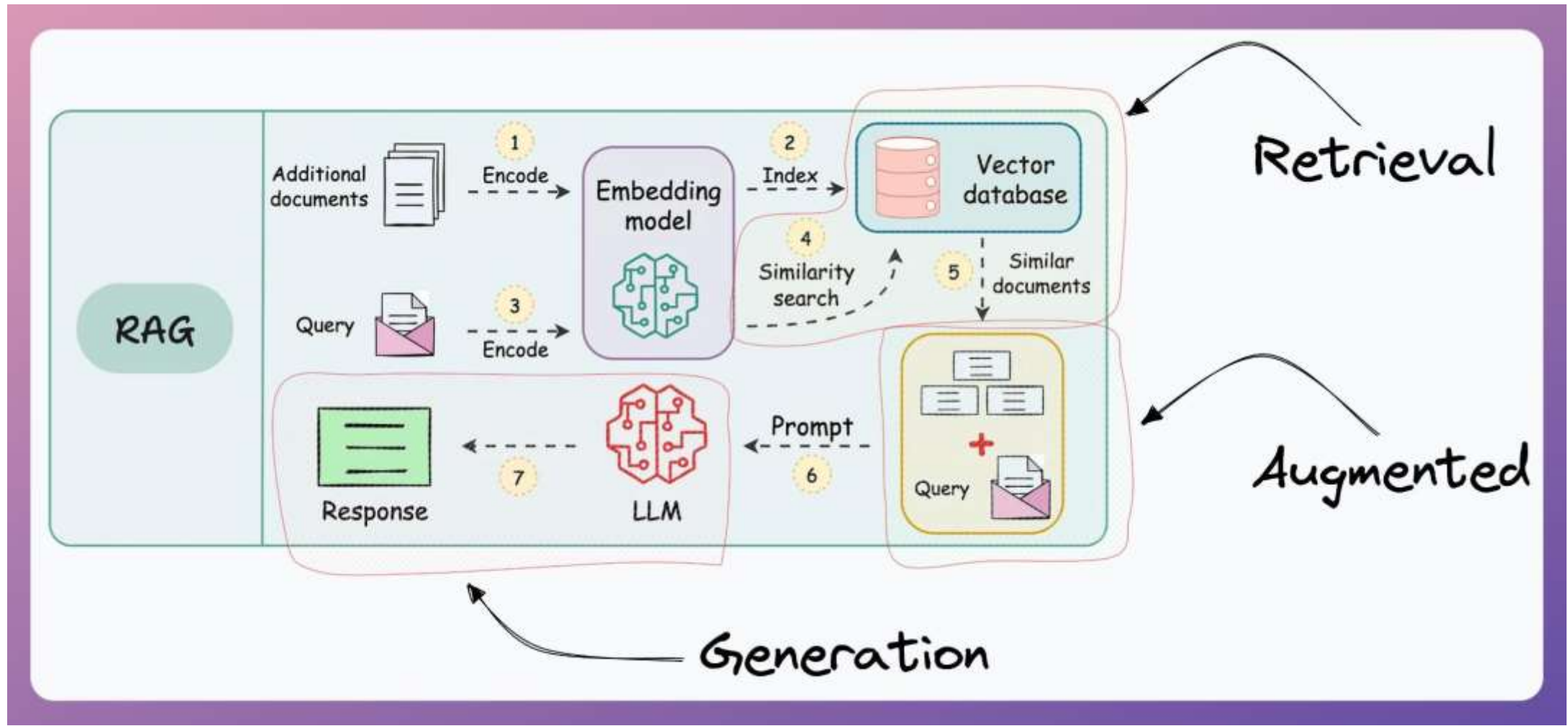
Each data point, whether a word, a document, an image, or any other entity, is transformed into a numerical vector using ML techniques.



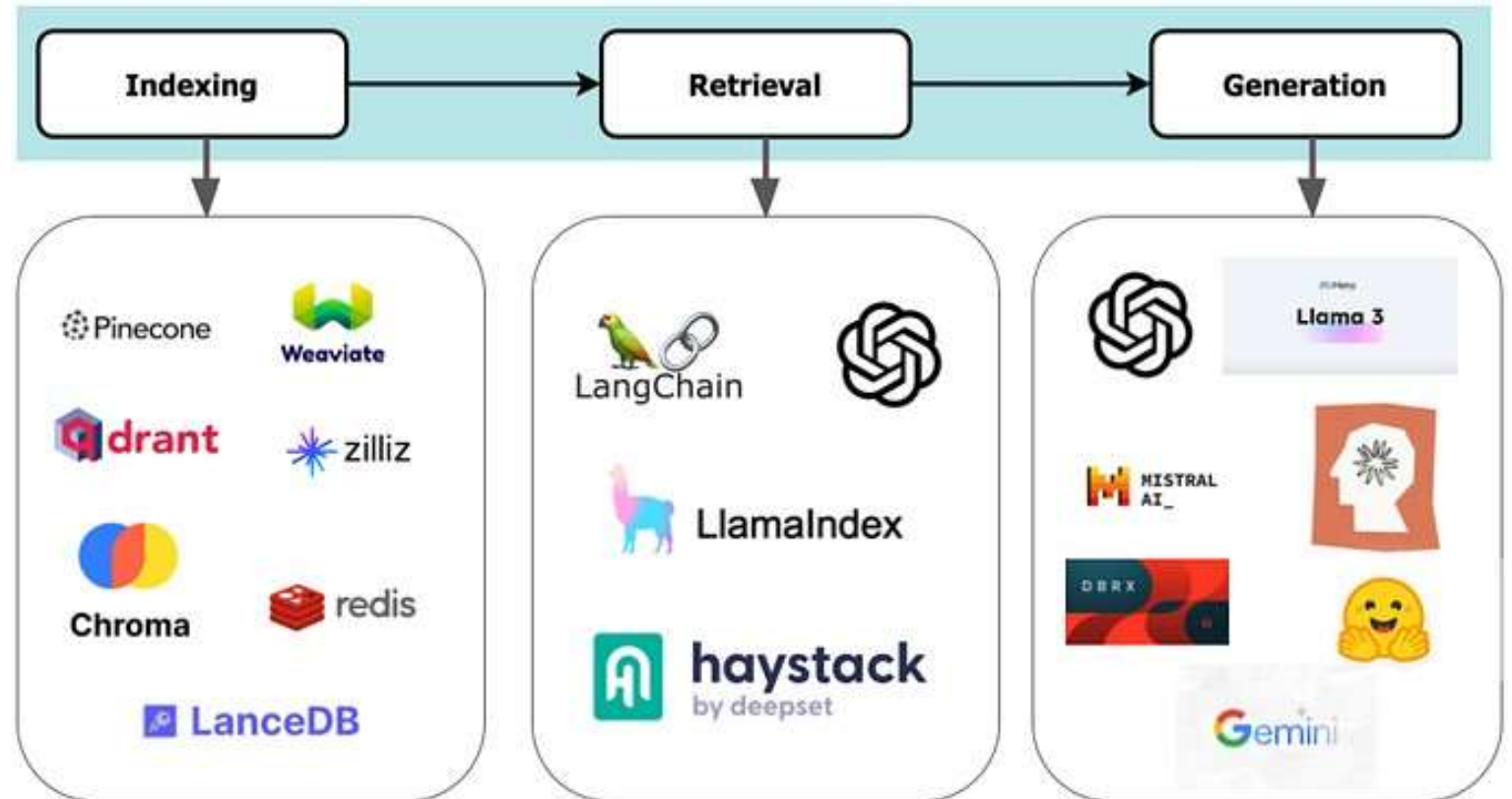
RAG – Putting all together



RAG re-represented

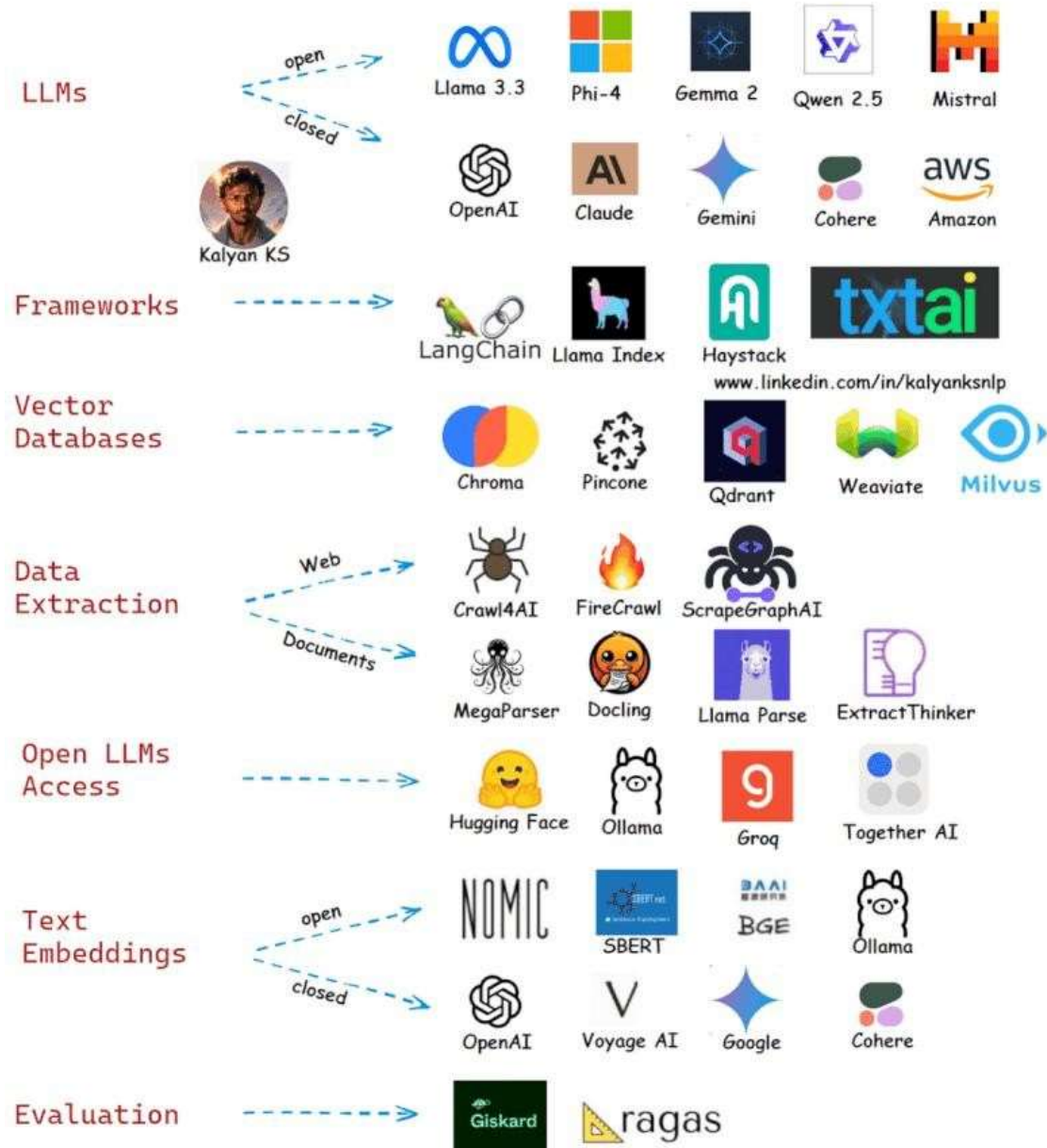


RAG developer's stack



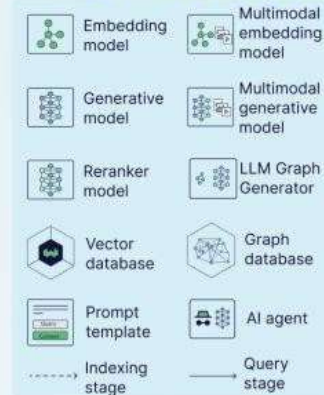
RAG developer' stack

RAG Developer's Stack

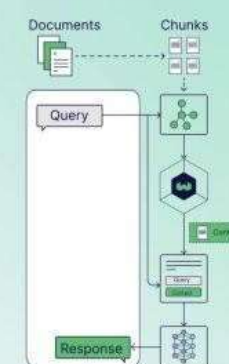


RAG Architectures

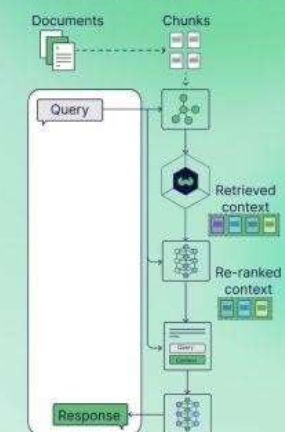
Retrieval-Augmented Generation Architectures



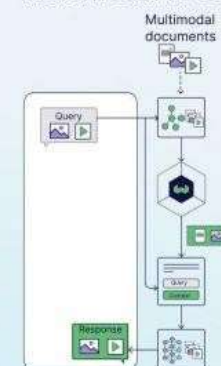
Naive RAG



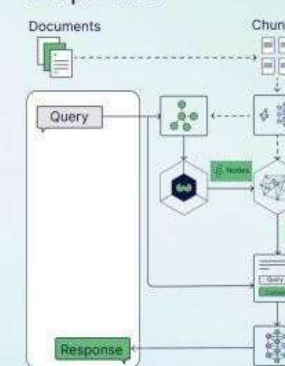
Retrieve-and-rerank



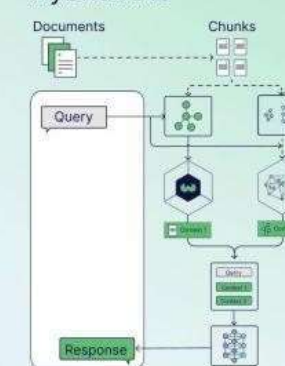
Multimodal RAG



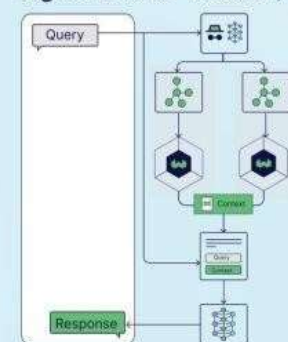
Graph RAG



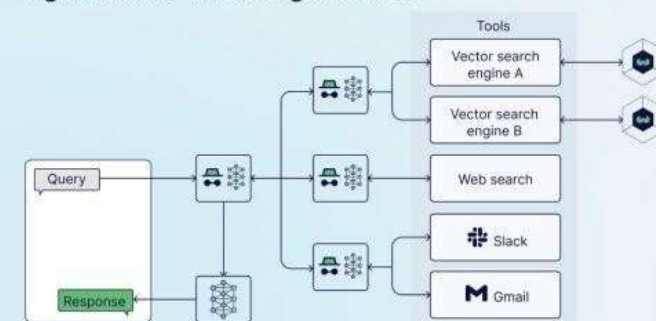
Hybrid RAG



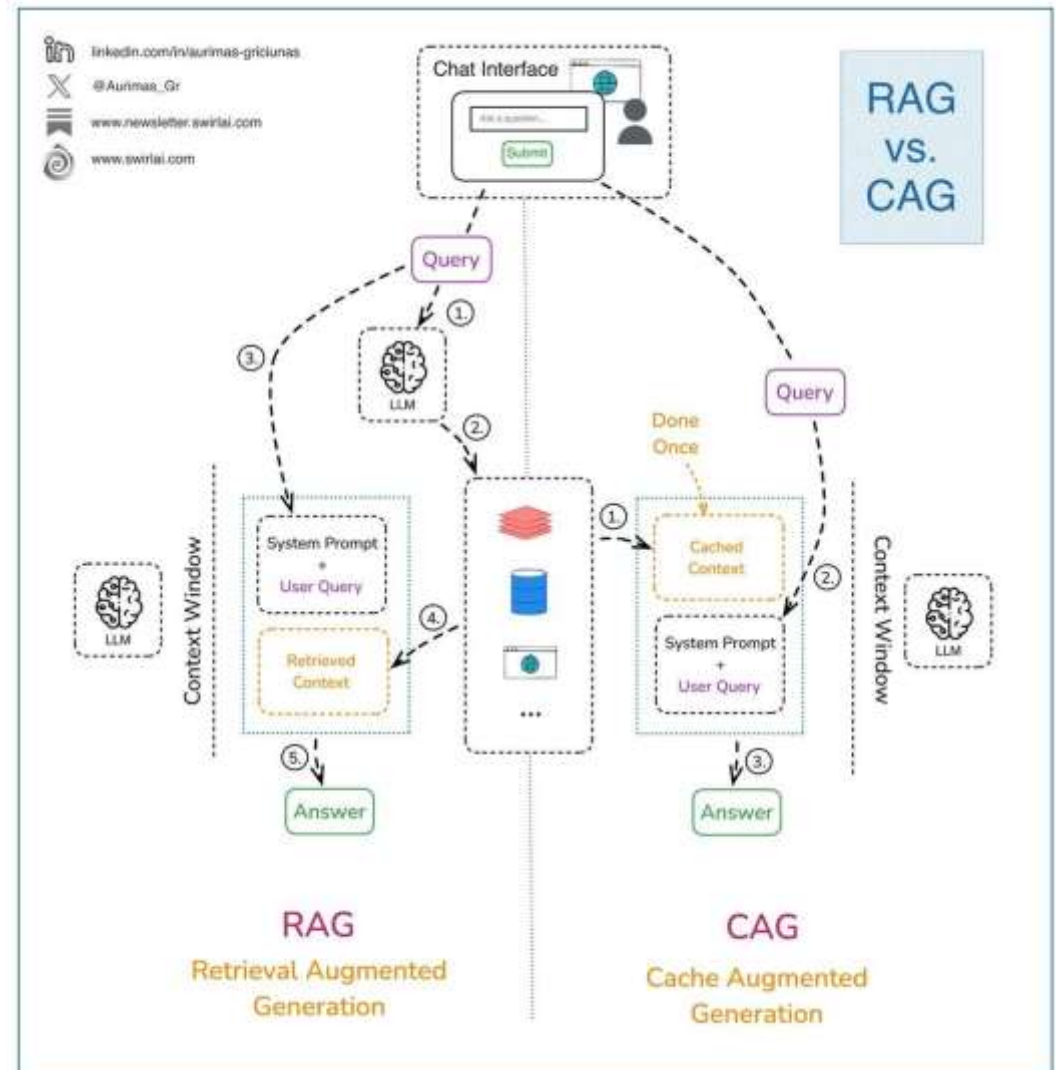
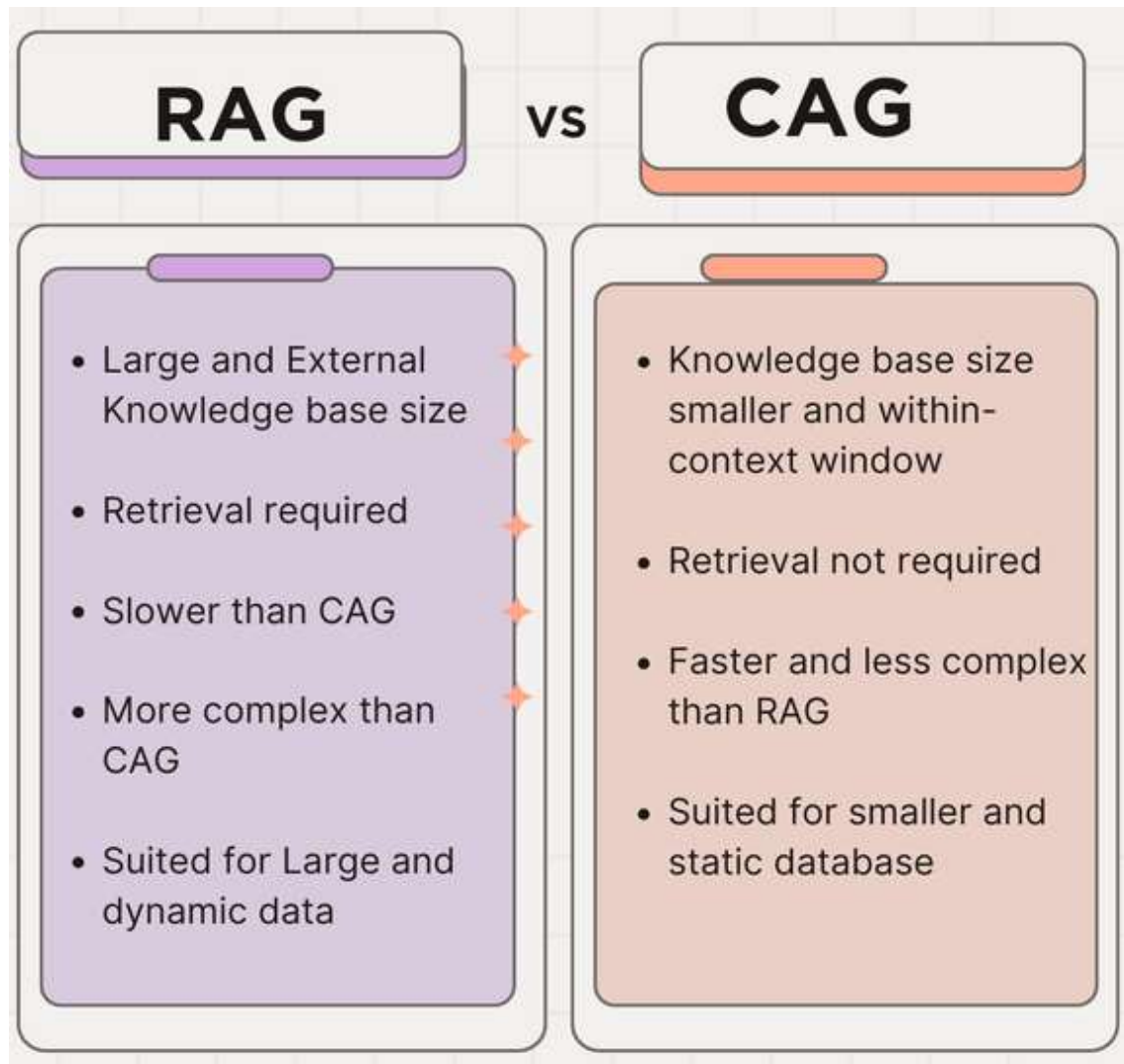
Agentic RAG (Router)



Agentic RAG (Multi-Agent RAG)



Cache-Augmented Generation (CAG)



Agentic frameworks

2025 AI Agent Tech Stack



<https://www.ibm.com/think/insights/top-ai-agent-frameworks>

Other Links

Apriora video Link:

<https://www.youtube.com/watch?v=cyCsNKhba0U>

To Create RAG System : [DeepLearning.AI](#)

Thank You

